

ISR C2 Platform — Interface Design Document (IDD)

Version	Date	Source of truth
0.4 (working draft)	2026-08-24	Working tree at HEAD b292a27

Status tags used throughout:

Tag	Meaning
<code>[live]</code>	Implemented in the current working tree.
<code>[partial]</code>	Partially implemented. Shape is stable, coverage is incomplete.
<code>[planned]</code>	Target contract. Not yet in code.

Where `[live]` and `[planned]` differ on a field name or shape, both are shown. The `[live]` form is what the system emits today.

Table of contents

Section	Title	Status
0	Conventions	<code>[live]</code>
1	System context and topology	<code>[live]</code>
IF-2	Site configuration and onboarding contract	<code>[live]</code>
IF-1	Sensor node to C2 ingest	pending
IF-3	Detection lifecycle and marker/aggregation contract	<code>[live]</code>
IF-4	Event data contract	pending
IF-5	Escalation, routing, tenant isolation	pending
IF-6	Receiver profile and interaction contract (v1)	<code>[live]</code> (Phase 2 shipped)
IF-7	Response and dispatch	pending
IF-8	Recording and evidence export	<code>[partial]</code> (trajectory scoping live)
IF-9	Agentic (Mistral) interfaces	<code>[partial]</code>
IF-10	Runtime API, DOM mount, external services	pending
A	Appendix: data dictionary, enums, ID formats, open gaps	pending

0. Conventions `[live]`

These conventions hold across every interface in this document unless a section overrides them explicitly.

0.1 Units

Quantity	Unit	Notes
Distance	metres (m)	All ranges, radii, offsets, and thresholds are metres unless a field name says otherwise (for example <code>maxChaseKm</code>).
Speed	metres per second (m/s)	Asset cruise speeds in the response catalog are the exception and use km/h in a field named <code>cruiseKmh</code> .
Altitude	metres above ground level (AGL)	<code>alt</code> and <code>altitude_agl_m</code> . An <code>altitude_msl_m</code> field mirrors AGL today and is reserved for future terrain correction.
Heading	degrees, 0 to 360, 0 = North, clockwise	Interpolation is wrap-aware (shortest arc).
Bearing (internal)	radians	Some interceptor internals carry <code>heading</code> in radians. Field-level notes call this out.
RF carrier	megahertz (MHz)	Integer channel centre, for example 2412, 5780.
RF power	dBm	Negative values, for example -68.
Acoustic level	dB	
Acoustic frequency	hertz (Hz)	
Confidence	float 0.0 to 1.0	Never a percentage in machine fields. Percentages appear only inside human-readable signature strings such as "OcuSync 91%" .

0.2 Coordinate order

This is the single most common integration mistake, so it is stated first.

- Point helpers and message fields use `(lat, lon)` order, and expose `lat` and `lon` as named fields.
- Polygon rings (`perimeter` , `siteBoundary` , `subLines`) use GeoJSON `[lon, lat]` order, longitude first.
- The great-circle distance helper is `haversineM(lat1, lon1, lat2, lon2)` and returns metres.

All coordinates are WGS84.

0.3 Time

Field pattern	Type	Meaning
<code>*_utc</code> , <code>timestamp</code> , <code>startTime</code> , <code>endTime</code> , <code>initiatedAt</code> , <code>at</code>	ISO 8601 string with <code>Z</code> suffix	Absolute UTC. Example <code>2026-08-23T14:22:07Z</code> .
<code>tSec</code> , <code>t_sec_from_event</code> , <code>tSec</code>	float seconds	Elapsed seconds since the event started. Fractional.
<code>spawnMs</code> , <code>dispatchedTs</code> , <code>*Ts</code> , <code>*Ms</code>	number, epoch milliseconds	Runtime timing. Not for persistence or cross-system correlation. Use the UTC fields for that.

0.4 Identifiers

Entity	Format	Example	Source
Event	<code>DET-YYYYMMDD-####</code>	<code>DET-20260823-0341</code>	<code>nextEventId()</code> in <code>events.js</code>

Entity	Format	Example	Source
Escalation	ESC-YYYYMMDD-####	ESC-20260823-0001	escalateEvent() in events.js
Dispatch instance	cd-{eventId}-{assetId}-{memberIndex}-{epochMs}	cd-DET-20260823-0341-flv-skryststrup-0-1690000000000	dispatch spawn in main.js
Dispatch group	cdg-{eventId}-{assetId}-{epochMs}	shared by all members of one dispatch	dispatch spawn in main.js
Site	lowercase slug	cph , esbjerg , energinet_amaager_koblingsstation	site config
Sensor	per-site short code	N01 , HVG-N01	site config
Drone key (within an event)	lead or sw{index}	lead , sw3	marker engine. index refers to state.swarmBillboards[index] .
Role	see IF-6	live IDs such as pet , politi , op-cph-airports . v1 target IDs such as rcv-pet .	roles.js

Identifiers are opaque to a partner. Do not parse them for meaning beyond the documented prefix.

0.5 Core enumerated vocabularies

Full enums live in the appendix. The vocabularies referenced most often:

Field	Values
status (event)	active , closed
classification	hostile , friendly , resolved (also unknown in rule conditions)
threat	high , medium , low , null (null when classification is not hostile)
platform	quadcopter , fixed-wing , jet , missile , helicopter , usv , null
status (sensor)	online , offline , degraded
kind (tenant/role)	admin , operator , receiver

0.6 Voice for generated human-facing text

Any human-facing string the platform generates, including model-generated narrative, follows house style:

- No em-dashes and no semicolons.
- Declarative voice. No hedge language, no filler.
- Plain ASCII, except Danish letters æ, ø, å which are always written in full and never transliterated to ae, oe, or aa.
- Detection-only framing. Never counter-drone, anti-drone, or kinetic language in partner-facing or receiver-facing text.

1. System context and topology [Live]

The current repository is the browser client. It carries the deterministic pipeline, the event and recording stores, and the agentic layer. That agentic layer is a Mistral client plus a two-agent site-context and correlation design (mistral.js, site_context.js, summary.js). The foundation is in place and accrues context as more detection instances are recorded, which is what lets the correlation agent cross-reference and analyse across events. What is not yet stood up is a hosted backend. There is no deployed server for persistence, real sensor-mesh ingestion, or a sovereign inference endpoint. The simulation runs in the browser and stands in for the sensor mesh a real deployment feeds in. The map engine is Cesium. The agentic interfaces are detailed in IF-9. Rendering constraints are in IF-10.

1.1 Tenancy model

Three tenant kinds. Strict isolation. Separate deployments in production. No cross-tenant data access.

Tenant	Who	What they buy	Access
Admin	ISR internal	Full platform. Provisions sensors, sites, rules, receiver mappings.	Everything.
Operator	Site owner (utility, port, grid operator, data centre, telecom, airport)	SaaS plus hardware-as-a-service. Sensors on their site plus the full C2 dashboard.	Live map, sensor health, event ledger, escalation to receivers, config, history, fleet. Scoped to their assigned <code>siteIds</code> .
Receiver	Government agency (Politi, PET, Forsvaret branches, Beredskabsstyrelsen, DSO)	SaaS only. Receives escalations from operators.	Filtered inbox scoped to their destination IDs. Brief detail, acknowledge, respond, cascade, dispatch coordination.

Isolation is enforced at the query layer, not only in the UI. A receiver sees only events escalated to a destination ID it owns. An operator sees only events at its assigned sites. Admin sees all. IF-5 specifies the enforcement functions. IF-6 specifies the full role registry and interaction rules.

1.2 Deployment topology

Three tiers. The edge tier is where a partner most often integrates.

Edge (sensor node). Sensor hardware for RF, acoustic, visual, and radar. A neural network runs on the node and performs modality processing, cross-modality fusion, and drone classification. The node emits fused detection output on a fixed tick, target cadence about 500 ms per node, configurable per site. Reference compute is Radxa Rock 4SE class. Signal-level processing lives here and is not a platform concern.

Client (operator and receiver browsers). Renders all UI. Runs the deterministic pipeline (correlation, recommendation, escalation routing, response coordination). Holds event registry, recording state, and timeseries. Calls a sovereign endpoint for model-generated narrative.

Sovereign backend (planned). Model inference on EU-hosted infrastructure. Real sensor mesh ingestion. Persistent event storage with signed evidence hashes. Cross-site correlation index. Multi-tenant deployment per customer.

1.3 The integration boundary

The platform is designed so real hardware plugs into the same downstream pipeline the mock data uses today. The seam is a single adapter interface, `NnOutputSource` . Everything upstream of that seam (raw signal, modality fusion, classification) is the sensor node's job. Everything downstream (event lifecycle, correlation, escalation, response coordination) is the platform's job.

The full adapter contract, the per-tick detection message, the sensor node descriptor, and the reconciliation between the live mock tick shape and the target schema are specified in IF-1.

1.4 Trust and isolation

- Sensor nodes push detection batches as trusted input. Field-hardware attestation is an open item, tracked in the appendix.
- Tenant data isolation is a hard guarantee, enforced at the query layer (IF-5).
- Model inference for narrative is sovereign by requirement. No US-hosted model calls in a government deployment. The current dev build calls a public model endpoint directly, which is a known pre-production gap noted in IF-9.

IF-2. Site configuration and onboarding contract [live]

A site is the top-level unit of deployment. Adding a site is the primary onboarding action and most of the platform auto-wires from a single site definition.

IF-2.1 Site object schema

`SITES[siteId]` is the canonical shape.

Field	Type	Required	Description
<code>id</code>	string	yes	Unique site slug. Matches the <code>SITES</code> key.
<code>name</code>	string	yes	Display name, for example <code>Copenhagen Airport</code> .
<code>code</code>	string	yes	Short code, for example <code>CPH</code> , <code>EBJ</code> , <code>HVG</code> .
<code>coordinates</code>	<code>{lat, lon}</code>	yes	Site centroid.
<code>subtitle</code>	string	no	Operational descriptor.
<code>operatorAccountId</code>	string	no	Owning operator account, for example <code>op-energinet</code> .
<code>perimeter</code>	<code>[[lon, lat], ...]</code>	conditional	Closed polygon ring, longitude first. The fenced yard. Drives ENTRY and EXIT markers.
<code>siteBoundary</code>	<code>[[lon, lat], ...]</code>	conditional	Fallback outer boundary used when <code>perimeter</code> is empty or absent.
<code>subLines</code>	<code>[[[lon, lat], ...], ...]</code>	no	Edge segments such as continuous quay lines, split on gaps over 250 m. Visual only.
<code>sensors</code>	<code>Sensor[]</code>	yes	Sensor nodes at this site. See IF-2.3.

Field	Type	Required	Description
<code>stats</code>	object	yes	Demo statistics: <code>{sensorsOnline, sensorsTotal, flaggedEvents24h, hostileEvents24h, falseAlarms24h}</code> .
<code>autoEscalateTiers</code>	number[]	see note	Tiers to auto-escalate to for events at this site. See IF-2.7. Promoted to required.

At least one of `perimeter` or `siteBoundary` must be present. If `perimeter` is empty the platform falls back to `siteBoundary` for polygon crossing tests.

IF-2.2 Configured sites

Eight sites are configured in the current tree.

Site ID	Name	Code	Centroid (lat, lon)	Sensors	Polygon source
<code>esbjerg</code>	Esbjerg Harbour	EBJ	55.467, 8.450	18	<code>siteBoundary</code> (27 vertices)
<code>cph</code>	Copenhagen Airport	CPH	55.618, 12.647	24	<code>perimeter</code> (74 vertices, OSM)
<code>energinet_hovegaard</code>	Hovegård Substation	HVG	55.73231, 12.23379	7	<code>perimeter</code> (15 vertices)
<code>energinet_bjaeverskov</code>	Bjæverskov HVDC Kontek	BJK	55.45151, 12.00729	7	<code>perimeter</code> (12 vertices)
<code>energinet_landerupgaard</code>	Landerupgård Substation	LDG	55.56398, 9.54762	7	<code>perimeter</code> (22 vertices)
<code>energinet_kassoe</code>	Kassø Substation	KAS	55.03682, 9.26960	7	<code>perimeter</code> (9 vertices)
<code>energinet_ferslev</code>	Ferslev Substation	FRV	56.95653, 9.87912	7	<code>perimeter</code> (17 vertices)
<code>energinet_amager_koblingsstation</code>	Amager Koblingsstation	AMK	55.6410, 12.6088	7	<code>perimeter</code> (5 vertices)

Core sites are defined in `sites.js`. The six Energinet substations are defined in `sites_energinet.js` and merged into `SITES` at load.

IF-2.3 Sensor object schema

Each entry in a site's `sensors` array.

Field	Type	Required	Description
<code>id</code>	string	yes	Sensor code, unique within the site, for example <code>N01</code> , <code>HVG-N01</code> .
<code>label</code>	string	yes	Human-readable name, for example <code>Tower Syd</code> , <code>ATC Core</code> .
<code>lat</code>	number	yes	Latitude, WGS84.
<code>lon</code>	number	yes	Longitude, WGS84.
<code>coverageRadius</code>	number	yes	Detection radius in metres. Currently 1000 for all nodes.
<code>status</code>	enum	yes	<code>online</code> , <code>offline</code> , or <code>degraded</code> . See coverage gate in IF-2.4.

Field	Type	Required	Description
<code>hardware</code>	string	yes	Platform string, for example <code>Radxa Rock 4SE + HackRF</code> .
<code>modalities</code>	string[]	yes	Declared modalities, for example <code>["RF", "Acoustic", "Visual"]</code> .
<code>detectionsLast24h</code>	number	no	Demo counter.
<code>issues</code>	string or null	no	Status note when degraded or offline, else null.
<code>isCore</code>	boolean	no	Marks the HackRF-equipped hub node. One per site.

IF-2.4 Coverage geometry model

Coverage is circular. Each online sensor defines a circle of radius `coverageRadius` metres centred at `(lat, lon)`. A point is covered by a site if it lies within any online sensor circle at that site. A point is covered globally if it lies within any online sensor circle at any site.

Helper	Signature	Returns	Purpose
<code>haversineM</code>	<code>(lat1, lon1, lat2, lon2)</code>	metres	Great-circle distance.
<code>_anySensorSeesPoint</code>	<code>(lat, lon)</code>	boolean	True if any online sensor at any site covers the point.
<code>_sitesSeeingPoint</code>	<code>(lat, lon)</code>	<code>Set<siteId></code>	The set of sites whose online sensors cover the point.
<code>nearestSensorInCoverage</code>	<code>(p, site)</code>	<code>{nearest, minDist, inCoverage}</code>	Site-local nearest sensor and whether the point is covered at that site.
<code>_shouldAutoDetect</code>	<code>(lat, lon)</code>	boolean	Detection gate. Today a thin wrapper over <code>_anySensorSeesPoint</code> . Reserved for future altitude gates, multi-sensor quorum, and modality quorum.

Coverage gate. A sensor participates in coverage when `status !== 'offline'`. A `degraded` sensor still contributes coverage. Only `offline` removes a sensor from the geometry.

Symbol visibility rule. A tracked object is drawn as a symbol if and only if `_shouldAutoDetect(lat, lon)` is true for its position. This holds for the lead drone, every wingman, and every event type. Outside all coverage an object is not drawn as a confirmed symbol. This rule is universal and is expanded in IF-3.

IF-2.5 Perimeter versus site boundary

Two polygon roles, both longitude-first rings.

- `perimeter` is the fenced yard. Crossing it drives ENTRY (outside to inside) and EXIT (inside to outside) markers. The crossing point is computed by `_segmentPolygonIntersection(prev, cur, perim)` to sub-metre precision, independent of tick rate.
- `siteBoundary` is the fallback used only when `perimeter` is empty or absent.

Coverage circles (IF-2.4) are a separate geometry from the perimeter polygon. A drone can be inside coverage while outside the fenced perimeter, and the reverse. Marker semantics for the two geometries are specified in IF-3.

IF-2.6 Critical-asset (target) model

Targets are points of interest that a drone trajectory can threaten. They are not sensor hosts and they do not belong to a single site.

Field	Type	Description
id	string	Unique target id, for example <code>amalienborg</code> , <code>meta-odense</code> .
name	string	Display name.
subtitle	string	Role description.
kind	enum	<code>royal</code> , <code>government</code> , <code>military</code> , <code>transport</code> , <code>data-centre</code> , <code>healthcare</code> , <code>broadcasting</code> , <code>financial</code> , <code>embassy</code> , <code>stadium</code> , <code>energy</code> .
lat	number	Latitude, WGS84.
lon	number	Longitude, WGS84.

Targets are independent geographic points. The platform measures trajectory proximity to them for projected-impact flagging. A curated set lives in `targets.js`. A larger high-voltage substation set is imported from OSM into `targets_hv.js`.

IF-2.7 Required fields for onboarding

For a site to route correctly, the following must be present. The scalability gap this closes: a site that spawns a cross-site linked event but carries no escalation tiers will escalate to nobody, silently.

Field	Requirement	Reason
<code>id</code> , <code>name</code> , <code>code</code> , <code>coordinates</code>	required	Identity and map placement.
<code>perimeter</code> or <code>siteBoundary</code>	at least one	ENTRY and EXIT geometry.
<code>sensors</code> with at least one <code>online</code> node	required	Otherwise the site never detects.
<code>autoEscalateTiers</code>	required (promoted from optional)	Without it, cross-site linked events at this site notify nobody. Must be a first-class field, not an optional.
<code>operatorAccountId</code>	required for tenant-scoped sites	Binds the site to an operator account for isolation.

IF-2.8 New-site onboarding checklist

Adding a `SITES` entry auto-wires the following with no further code:

- Coverage and detection helpers, global and site-scoped.

- Contributing-sensor population on live events.
- Per-site event lifecycle and marker firing for both boundary types (IF-3).
- Sensor point-of-view, sensor popup, and coverage rings.
- Swarm-density aggregation.

The following remain manual per site today and are non-blocking for real ingest:

- Fly-to buttons and the Simulate Threat selector in index.html.
- Threat templates in drones.js (simulation only, not needed for live sensor input).

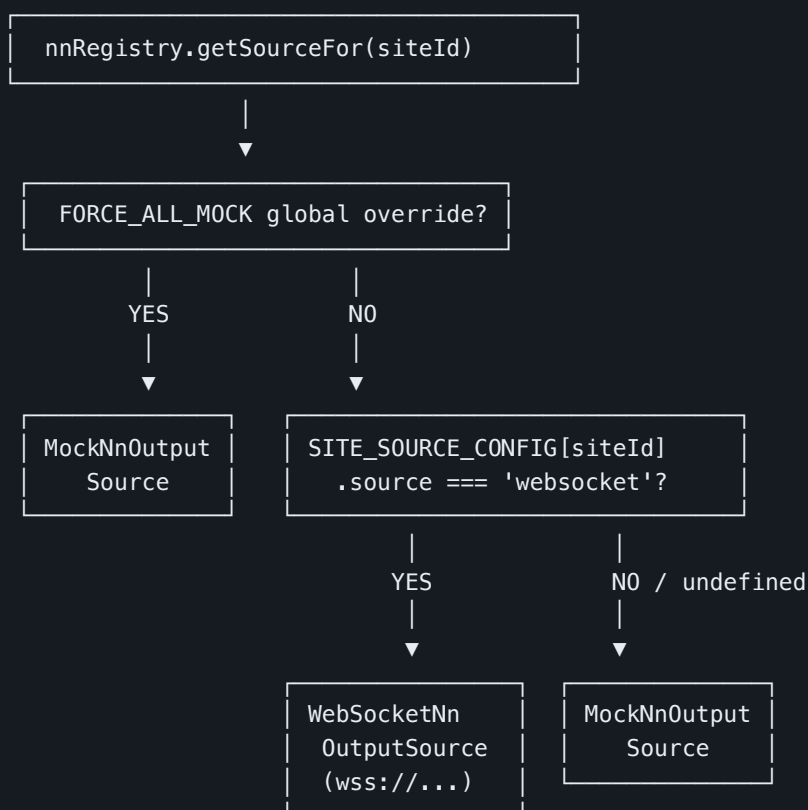
IF-1. Sensor node to C2 ingest [live]

The C2 platform is source-agnostic. Every downstream layer (tick loop, event lifecycle, correlator, UI, agents) consumes detections from an abstract `NnOutputSource` interface. Two implementations ship today: `MockNnOutputSource` (drives every simulation, demo, training run, and internal test) and `WebSocketNnOutputSource` (consumes real field sensor node streams). A per-site registry decides which source owns which siteId. This is the single seam through which real hardware plugs in without touching downstream code.

The simulation layer (drones.js, TEMPLATES, waypoint interpolation, swarm formations) is **not** replaced by real hardware. It is a permanent, first-class subsystem — used for demos, training, edge-case regression, and side-by-side sim/real customer visits. The registry decides per-site which detection stream reaches the tick loop; the simulation engine keeps feeding the mock source at whatever sites route to it.

IF-1.1 Source-selection decision tree

Per detection tick, for each siteId:



Route matrix for every operating mode:

Mode	FORCE_ ALL_MOCK	Per-site config	Result
Pure simulation / demo recording	true	ignored	Every site → mock. Simulation-engine drones flow through mock source as today.
Mixed sim + live (customer visit, staged rollout)	false	some sites <code>websocket</code> , others <code>mock</code> (or undefined)	Websocket sites consume real sensor streams; mock sites keep running scripted templates on the same map.
Full production	false	every site <code>websocket</code>	Every site consumes real hardware. Mock code stays in the bundle (available for on-demand replays and post-incident sim).
Development default	false	all undefined	Every site → mock. Zero-config behaviour matches today's build byte-for-byte.

IF-1.2 `NnOutputSource` interface

```
interface NnOutputSource {
  siteId: string
  start(): void
  stop(): void
  onDetection(callback: (batch: NnDetectionBatch) => void): void
  ingestPositions(positions): void // simulation push path, real sources ignore it
}
```

- `start()` / `stop()` control the source's lifecycle. Mock starts a no-op subscription to the sim tick; `WebSocket` opens the connection. Both are idempotent.
- `onDetection(cb)` registers a listener. Sources may emit many callbacks per tick (one per detection); the tick loop batches downstream.
- All emitted batches share the same schema (IF-1.4). The tick loop, correlator, and UI cannot tell which source the batch came from.
- `ingestPositions(positions)` is the simulation push path. The tick loop calls it each tick with the current drone positions for the site. The mock source runs the coverage math inside it and emits a batch. A real source ignores it, because real batches arrive over the wire (`main.js` calls it on every source at line 5221).

IF-1.3 Implementations

`MockNnOutputSource(siteId)` — reads live sim state (`SITES[siteId].sensors` + current drone positions per event from the simulation engine), computes per-sensor coverage + confidence exactly as `_updateContributingSensorsForPosition` does today, and emits an `NnDetectionBatch` per tick. This is the same math the C2 currently runs inline; the extraction is pure-refactor with no behavioural change. Keeps every simulation feature — waypoint interpolation, swarm formations, RF signature generation, template scenarios — permanently functional.

`WebSocketNnOutputSource(siteId, url, opts)` — connects to a real field sensor node stream.

Contract:

- Reconnect with exponential backoff on drop (base 1 s, max 30 s).

- On disconnect, mark all sensors at that site `status: 'offline'` in the emitted batch so downstream degradation logic is uniform between mock and real sources.
- Accept a bearer token via `opts.auth` for authenticated node streams.
- Enforce site-scoped delivery: reject batches whose `siteId` field does not match this source's `siteId` (prevents cross-site injection from a compromised node).

Both implementations live in `src/nn_source.js`. The registry (`src/nn_registry.js`) owns the per-site source map + the `FORCE_ALL MOCK` global.

IF-1.4 `NnDetectionBatch` shape

```
type NnDetectionBatch = {
  siteId: string
  tickTs: string // ISO-8601 wall clock of the emitting tick
  sensors: Array<{
    sensorId: string // must exist in SITES[siteId].sensors
    status: 'online' | 'offline' // per-tick health from the source
    detections: Array<{
      lat: number
      lon: number
      alt?: number
      confidence: number // 0..1 in the emitting source's frame
      droneKey?: string // 'lead' | 'sw{i}' | opaque real-hardware ID
      modalityMix?: string[] // ['RF','Acoustic','Visual'] subset that fired
    }>
  }>
}
```

Same shape from mock and real sources. Downstream code aggregates by (`siteId`, `sensorId`) with per-tick MAX across drones (the same monotonic-tick reset behaviour landed in `_updateContributingSensorsForPosition`).

IF-1.5 Configuration surface

Global override (`env / bootstrap`):

```
window.__ISR_FORCE_ALL MOCK = true // wins over per-site config; used for
// demo recording sessions and offline
// simulation runs
```

Per-site (`src/nn_registry.js`):

```
SITE_SOURCE_CONFIG = {
  // Every site ships against mock today. No field hardware is wired yet.
  cph: { source: 'mock' },
  esbjerg: { source: 'mock' },
  amk: { source: 'mock' },
  // Flip a site to live when its field node ships, for example:
  // cph: { source: 'websocket', url: 'wss://cph-node-01.field.isr/nn', auth: '<token>' }
  // Sites not listed default to mock, so a new site in SITES works with no registry update.
}
```

IF-1.6 Migration + rollback contract

- Adding a real sensor node to a site is one registry entry. Zero downstream code changes.

- Rolling back a misbehaving field deployment: change the site's registry entry to `{ source: 'mock' }`. Downstream code keeps running against the mock source until the field is stable again.
- Adding a whole new site: create the entry in `SITES` (sensors, coordinates, receivers, boundary). The registry defaults to mock; live wiring is added when the field node ships.
- Multi-source composition (Phase 4 of the agentic architecture doc) is supported by construction — each site's source is independent.

IF-1.7 What this unlocks

- Netcompany integration can plug in as a `WebSocketNnOutputSource` variant (or a sibling source that consumes their aggregated feed) without touching the tick loop.
- Customers running their own detection stack can write an adapter that maps their output to `NnDetectionBatch` (IF-1.4). Their infra plugs into the same downstream pipeline.
- New NN model versions can be A/B tested at the adapter layer without redeploying the C2.
- Sim-only demo installs (trade shows, sales demos, training environments) run the same C2 build with `FORCE_ALL MOCK = true`.

IF-3. Detection lifecycle and marker/aggregation contract [Live]

The detection lifecycle contract defines how the platform turns per-drone sensor coverage transitions into on-map markers and audit records. It is coordinate-only (no threat classification here) and is the single source of truth for the visual timeline of any event. Every rule below is enforced per-drone and aggregated at event level.

Detections reach this layer as `NnDetectionBatch` batches from the source in IF-1. This section covers what happens to the coverage transitions inside those batches, not where they come from. The source is abstract, so the rules below run identically whether a detection came from the mock source or a real field node.

IF-3.1 Design invariants

Five invariants hold across every event type, every site, every drone.

1. **Symbol visibility is coverage-gated.** A drone is drawn as a symbol if and only if `_shouldAutoDetect(lat, lon)` returns true. This holds for the lead drone, every wingman, and every event type (IF-2.4).
2. **Boundary crossings drive markers.** There are two boundary geometries per site: the fenced-yard `perimeter` polygon and the sensor coverage circles. Each fires its own marker type on transition.
3. **Coordinates are exact.** Every marker is placed by segment-boundary bisection using the drone's own `prev→cur` segment. 22 iterations of bisection give sub-metre precision independent of tick rate. No shared anchor, no stale billboard reads, no anchor swap discontinuities.
4. **Swarm-density decides aggregation.** If the drone firing a transition is within `SWARM_PROXIMITY_M` (1000 m) of any other live drone in the same event, the marker only fires

on the aggregate empty↔non-empty flip. Otherwise it fires per-drone with the drone identity in the label.

5. **Death is silent.** When a drone dies (interceptor kill, other), it is purged from every aggregate set without firing a marker. If the purge empties the aggregate and OOR has not fired for the current cycle, a delayed OOR fires at the position where the surviving drone actually exited coverage (recorded on `agg._pending0orPos`).

IF-3.2 Entry point

The full per-tick contract is one function.

```
processDroneSiteMarkers(event, state, droneKey, curPos, prevPos, droneLabel)
```

Argument	Type	Notes
<code>event</code>	Event	The event this drone belongs to.
<code>state</code>	droneState	The runtime state bag for the event (used for live-count filtering on death).
<code>droneKey</code>	string	'lead' or <code>`sw\${index}`</code> where <code>index</code> refers to <code>state.swarmBillboards[index]</code> .
<code>curPos</code>	<code>{lat, lon}</code>	This drone's current tick position.
<code>prevPos</code>	<code>{lat, lon}</code> or null	This drone's previous tick position. Null on the first tick this drone was processed.
<code>droneLabel</code>	string	Human-readable identity for individual-mode markers (for example <code>Autel EVO II Pro (unknown link)</code>).

The function is called from the swarm loop for every live wingman and from the lead position callback for the lead. It never touches the marker engine's shared state via a global anchor.

IF-3.3 Boundary types and marker semantics

Two boundaries per site fire four marker kinds.

Marker	Trigger	Boundary	Colour
<code>ENTRY</code>	outside → inside	<code>perimeter</code> polygon (or <code>siteBoundary</code> fallback)	<code>#4dd2ff</code> blue
<code>EXIT</code>	inside → outside	<code>perimeter</code> polygon	<code>#ffb84d</code> yellow
<code>DETECTED</code>	out-of-cov → in-cov, first ever this event this site	union of online sensor circles	<code>#4dd2ff</code> blue
<code>REACQUIRED</code>	out-of-cov → in-cov, subsequent	union of online sensor circles	<code>#4dff9c</code> green
<code>OUT OF RANGE</code>	in-cov → out-of-cov	union of online sensor circles	<code>#ff5a5a</code> red

Coordinate placement.

Marker	Bisection helper	Fallback if segment doesn't straddle
ENTRY / EXIT	<code>_segmentPolygonIntersection(prev, cur, perim)</code>	Current position.

Marker	Bisection helper	Fallback if segment doesn't straddle
DETECTED / REACQUIRED / OUT OF RANGE	Inline 22-iteration segment-circle bisection against every online sensor at the site	For OOR, <code>agg._pending0orPos</code> if recorded, else current position.

IF-3.4 Per-drone per-site state

Each drone owns its own boundary state per site. Two maps are held on the event for every drone.

```
event._droneCovState:  Map<droneKey, Map<siteId, boolean wasInCov>>
event._droneInsideState: Map<droneKey, Map<siteId, boolean wasInside>>
```

`wasInCov` reflects the drone's last observed relationship to that site's sensor coverage.

`wasInside` reflects the drone's last observed relationship to that site's `perimeter` polygon.

Both are set by `processDroneSiteMarkers` at the end of each tick per site.

No shared anchor exists. Anchor swap when a drone dies is not a concept in this model. Every drone contributes its own transitions.

IF-3.5 Event-level aggregation

Aggregation collapses many drones into one marker per (site, boundary type) when the drones are close together.

```
event._siteAgg[siteId] = {
  insideDrones:  Set<droneKey>,    // membership: drones currently inside perimeter
  inCovDrones:   Set<droneKey>,    // membership: drones currently in coverage
  entryCount:    number,           // aggregated ENTRY markers fired
  exitCount:     number,           // aggregated EXIT markers fired
  detectCount:   number,           // aggregated DETECTED / REACQUIRED markers fired
  oorCount:      number,           // aggregated OUT OF RANGE markers fired
  droneEntryCount: Map<droneKey, number>, // per-drone individual-mode counts
  droneExitCount:  Map<droneKey, number>,
  droneDetectCount: Map<droneKey, number>,
  droneOorCount:   Map<droneKey, number>,
  _pending0orPos:  {lat, lon} | null, // remembered exit point for delayed OOR
  _oorFiredThisCycle: boolean,       // gate against double-fire per cycle
}
```

Membership semantics. `insideDrones` and `inCovDrones` are Sets keyed by `droneKey`. A drone is added on its out→in transition and removed on its in→out transition. Aggregate counts increment when a marker actually fires.

Live-count check. The "aggregate empty" check filters dead drones on the fly via a live-count walk. A drone key stays in the Set until purged, so if a drone dies while still recorded as "in coverage" the Set is not immediately empty. The live-count check ignores dead keys so the empty flip fires correctly. The dedicated helper is inlined in the marker function as `_liveOnly(set)`.

IF-3.6 Swarm-density aggregation

The gate that decides swarm mode versus individual mode.

```
_isDroneInSwarm(state, thisDroneKey, thisPos) -> boolean
```

Returns true if any OTHER live drone in the event is within `SWARM_PROXIMITY_M` metres of `thisPos`. Live means non-neutralised in either `state.leadSwarmMember` or `state.swarmBillboards[i]`. The threshold is 1000 m and is the single tunable knob for this behaviour.

Rule applied inside `processDroneSiteMarkers` :

Mode	When	Marker firing
Swarm (<code>inSwarm=true</code>)	Any other live drone within 1000 m	Aggregate empty↔non-empty flip fires one marker. Label omits drone identity.
Individual (<code>inSwarm=false</code>)	This drone is isolated from all other live drones	Fires a per-drone marker labeled with <code>droneLabel</code>

The gate is evaluated on every tick per drone. A drone that is in swarm mode during ingress and individual mode during escape sees both behaviours during a single event.

IF-3.7 Death handling

Two functions fire on the alive→dead tick per drone.

```
_purgeDroneFromAggregates(event, droneKey)
```

Removes the drone key from every site's `insideDrones` and `inCovDrones` Sets. Silent (no marker). The drone was killed by response, not signal-lost. Called once per drone (guarded by `sw._deathPurged` for wingmen and `state.leadSwarmMember._deathPurged` for the lead).

```
_fireDelayedOORForEmptiedSites(event, state, deathLat, deathLon)
```

After the purge, sweeps every site aggregate. For any site whose `inCovDrones` is now empty of live drones AND `_oorFiredThisCycle` is false, fires OOR:

- at `agg._pendingOorPos` if recorded (the position where the surviving drone actually left cov earlier, remembered on that drone's own out-of-cov transition),
- else at `(deathLat, deathLon)` (the dying drone's death position, fallback).

Sets `_oorFiredThisCycle = true` and clears `_pendingOorPos`. `_oorFiredThisCycle` is reset to false on the next cov-gain transition so the next OOR cycle can fire.

This closes the exact scenario where a fleeing drone exits AMK coverage while wingmen are still alive: no OOR fires at that moment (aggregate non-empty), the exit position is remembered on `_pendingOorPos`, then when the last wingman dies the delayed OOR fires at the fleeing drone's earlier exit point.

IF-3.8 Notification pipeline

OUT OF RANGE fires an operator-facing toast at every aggregate empty transition, including delayed OOR. Toast copy:

```
OUT OF RANGE · <droneType or droneLabel> left <site.name or site.code> sensor coverage.
```

Individual-mode OOR fires the toast per drone with the drone label. Swarm-mode OOR fires once per aggregate empty. DETECTED, REACQUIRED, ENTRY, and EXIT do not currently emit toasts (retained for consistency once IF-5 notification pipeline lands).

IF-3.9 Audit feed versus render feed

Two separate data feeds are populated by every transition.

Feed	Field on event	What it captures	Downstream consumers
Audit	<code>event._droneTransitionLog</code>	Every transition, always logged, regardless of aggregation mode. <code>{droneKey, droneLabel, siteId, kind, lat, lon, timestamp}</code> .	Per-drone summary reports, forensic timeline, model briefing prompts.
Render	<code>event.perSiteCrossings</code>	Only markers that actually fired on the map. <code>{kind, lat, lon, color, label, timestamp}</code> .	Map rendering, PDF export, JSON and CSV evidence, re-select restore.

The split is deliberate. Aggregation collapses map markers for UX clarity, but every drone's true transition history remains preserved in the audit feed for reports and compliance.

IF-3.10 Frame-of-reference and integration boundary

The marker engine reads sensor coverage from `SITES` via the helpers in IF-2.4. It reads per-drone positions from `droneState` populated by the tick loop. It writes only to `event._siteAgg`, `event._droneCovState`, `event._droneInsideState`, `event._droneTransitionLog`, `event.perSiteCrossings`, and the Cesium marker entities in `_perEventMarkers`.

A real sensor mesh integrates by populating `droneState` on tick — the marker engine has no coupling to the simulator. When IF-1 lands (sensor node to C2 ingest), its adapter feeds the same `droneState` shape and the marker engine runs unchanged.

IF-3.11 Legacy shim

`processPerSiteMarkers(state, p, eventId, prevOverride)` is a no-op shim retained for call-site compatibility. All logic lives in `processDroneSiteMarkers`. Do not add new callers of the shim.

IF-4. Event data contract

Status: pending. Will specify the event object, the DetectionSubject canonical shape, droneState runtime schema, and multi-site shadow linking.

IF-5. Escalation, routing, tenant isolation

Status: pending. Will specify destinations, the rules engine, the escalation payload, and the query-layer isolation functions.

IF-6. Receiver profile and interaction contract (v1) [[live through Phase 2]]

The receiver contract centralises how government and organisational profiles interact with events on the platform. Prior to this contract every role-scoped decision (which receivers get notified, which CTAs render, who can hand off to whom) was scattered across inline `if role === 'politi'` branches, hardcoded destination arrays, and per-site rules. The contract folds all of that into one registry + a small set of pure helpers so a new receiver plugs in by adding a row.

Status split. The role registry + relationship helpers + `canInitiate` gate + `impactedRoles` helper are live and testable in the browser console. The consumer-side wiring (`SITES.receivers` block, `availableCTAsForReceiver`, `pushNotification` wrapper, observer UI) is planned per the migration path in IF-6.12.

IF-6.1 Design principle

Three properties the contract preserves.

- 1. **Pluggability.** Adding a receiver is a data change (a row in `roles.js`). No code changes to the marker engine, event lifecycle, or UI rendering.
- 2. **Explicit relationships.** Every role's position in the Danish government hierarchy is machine-readable via `parentId`. The `relationshipBetween` helper classifies any pair (self / parent / child / sibling / same-branch / cross-branch). This drives which flow types are allowed.
- 3. **Actor versus observer separation.** Any role can be looped into any event as an observer (in-the-loop, no CTAs). Observers can request promotion to actor when they need to act. Prevents "notification spam" on wide loops while keeping cross-agency awareness intact.

IF-6.2 Role registry

The registry is a flat `RECEIVERS` array in `roles.js` with lazy parent-chain resolution. Every entry has the following minimum shape.

Field	Type	Required	Description
<code>id</code>	string	yes	Unique role slug. Kebab-case. See naming convention below.
<code>kind</code>	enum	yes	<code>admin</code> , <code>operator</code> , <code>receiver</code> .
<code>type</code>	enum	yes	<code>leaf</code> (a specific unit) or <code>parent</code> (a branch node with <code>childrenIds</code>).
<code>org</code>	string	yes	Danish name with æ, ø, å.
<code>label</code>	string	yes	Display label in the UI. Usually matches <code>org</code> or expands it.
<code>initials</code>	string	yes	2-3 letter monogram for compact chips.
<code>scope</code>	enum	yes	<code>all-sites</code> , <code>national</code> , <code>regional</code> , <code>aviation</code> , <code>maritime</code> , or a specific site slug.

Field	Type	Required	Description
<code>destinationIds</code>	string[]	yes	Legacy destination IDs (<code>{siteCode}-t{tier}-{roleSlug}</code>) that route escalations to this receiver. Empty array on new sourced entries; populated as sites declare them (IF-6.8).
<code>parentId</code>	string	no	Role id of this leaf's direct parent. Null for top-level branches.
<code>childrenIds</code>	string[]	required on <code>parent</code> type	Role ids of this branch's direct children.
<code>description</code>	string	yes	One-line human description.
<code>person</code> / <code>logo</code>	string	no	Optional display fields for demo.
<code>meta</code>	object	no	Branch-specific fields, e.g. <code>region</code> , <code>region_parent</code> , <code>member_kommune</code> r .

Naming convention (id scheme).

<code>op-{operator-slug}</code>	<code>rcv-agency-{slug}</code>
<code>rcv-min-{ministry-slug}</code>	<code>rcv-pol-special-{unit}</code>
<code>rcv-pol-{district-slug}</code>	<code>rcv-forsvar-{branch}-{unit}</code>
<code>rcv-forsvar-hq</code>	<code>rcv-brs-special-{slug}</code>
<code>rcv-brs-{center-slug}</code>	<code>rcv-hjv-distrikt-{slug}</code>
<code>rcv-hjv-{region}</code>	<code>rcv-region-ambu-{region}</code>
<code>rcv-region-{region-slug}</code>	
<code>rcv-hospital-{slug}</code>	
<code>rcv-kom-{kommune-slug}</code>	<code>rcv-kom-berskab-{shared-slug}</code>

Legacy shorter ids (`politi` , `beredskab` , `flv-skrydstrup`) coexist for backwards compatibility. A shim (`ROLE_ID_ALIAS`) resolves deprecated ids like `flv-qra` → `flv-skrydstrup` so old call sites do not break during migration.

Registry cardinality at v0.2 (HEAD 1b9b909).

Branch	Count	Notes
Admin	1	ISR Systems.
Operators	~4	Site-owner accounts (CPH, Esbjerg, Energinet, others as onboarded).
Ministries	5	Justits, Forsvar, Klima, Erhverv, Sundhed.
Agencies (styrelser)	4	Trafik, Søfart, Energi, CFCS.
Politi	15	Rigspolitiet + 12 politikredse + NSK + AKS + PET.
Forsvaret	21	HQ + Flyvevåbnet (5) + Hæren (12) + Søværnet (5) + SOK + intel (FE) + Cyber Kommandoen.
Beredskabsstyrelsen	8	HQ + 5 centre + 2 specialty (Kemisk, Nukleart).
Hjemmeværnet	19	HQ + Vest + Øst + Marine HQ + Flyver HQ + 12 Hær distrikter (Marine/Flyver flotiller and eskadriller captured in supplementary comment).
Regioner	5	Hovedstaden, Sjælland, Syddanmark, Midtjylland, Nordjylland.
Akuthospitaler	23	24/7 acute-reception sites across all regioner.

Branch	Count	Notes
Kommuner	98	Full national set. Each has <code>meta.region</code> .
Kommunale beredskaber	29	Shared municipal fire/rescue services. Each has <code>meta.member_kommuner</code> .
Forsvarskommandoen (legacy)	1	Retained.
Total receivers	242	Merged into <code>roles.js</code> on 2026-08-23 (commit <code>410a68a</code>).

Data population TODO: additional Forsvaret named units, per-flotille and per-eskadrille granularity under Marinehjemmeværnet and Flyverhjemmeværnet if per-unit routing is required (captured in supplementary comment blocks in `scratchpad/roles-sourced-data.js`).

IF-6.3 Taxonomic tree

Top-level branches used for cross-branch versus same-branch classification.

Constant	Root role	Covers
<code>AGENCY_BRANCHES.POLITI</code>	<code>politi</code>	Rigspolitiet, 12 politikredse, PET, NSK, AKS.
<code>AGENCY_BRANCHES.FORSVARET</code>	<code>forsvaret</code>	Flyvevåbnet, Hæren, Søværnet, SOK, FE, CFCS, Cyber Kommandoen.
<code>AGENCY_BRANCHES.BRS</code>	<code>brs</code>	BRS HQ, 5 beredskabscentre, specialty units.
<code>AGENCY_BRANCHES.HJV</code>	<code>hjb</code>	Hjemmeværnet regions + branches + distrikter.
<code>AGENCY_BRANCHES.REGION</code>	<code>region-*</code>	Each region is its own top-level entity.
<code>AGENCY_BRANCHES.MINISTRY</code>	<code>min-*</code>	Ministries.
<code>AGENCY_BRANCHES.AGENCY</code>	<code>agency-*</code>	Styrelser under any ministry.
<code>AGENCY_BRANCHES.STANDALONE</code>	<code>miscellaneous</code>	Legacy or standalone nodes (<code>beredskab</code> , <code>forsvarskmd</code>).

The resolver walks `parentId` to the root and buckets the result into one of these constants.

```
agencyBranchOf(role_id) -> AGENCY_BRANCHES constant | null
```

Cached per lookup for O(1) repeat calls.

IF-6.4 Relationship classification

Any two role ids resolve to one of six relationship classes.

```
relationshipBetween(aId, bId) -> 'self' | 'parent' | 'child' | 'sibling' | 'same-branch' | 'cross-branch'
```

Class	Meaning
<code>self</code>	Same role id.
<code>parent</code>	A is parent of B (A's <code>parentId</code> === B's id).
<code>child</code>	A is child of B.
<code>sibling</code>	Same <code>parentId</code> , both non-null.

Class	Meaning
same-branch	Same <code>agencyBranch</code> but not direct parent/child/sibling. Example: Vestegnets Politi ↔ PET (both under Politi via different paths).
cross-branch	Different <code>agencyBranch</code> . Example: Politi ↔ Flyvevåbnet.
unknown	One or both role ids not found.

Examples verifiable in the browser console via `__isrRoles.relationshipBetween(...)`.

```

relationshipBetween('politi-vestegn', 'politi-nordsj')    -> 'sibling'
relationshipBetween('rigspoliti', 'politi-vestegn')      -> 'parent'
relationshipBetween('politi-kbh', 'flv-skrydstrup')      -> 'cross-branch'
relationshipBetween('brs-hedehusene', 'brs-herning')    -> 'sibling'
relationshipBetween('pet', 'politi-vestegn')             -> 'same-branch'

```

IF-6.5 Flow types

Canonical flow taxonomy for role-to-role interactions. Exported as `FLOW_TYPES`.

Constant	Value	Meaning
NOTIFICATION	notification	System-generated push, no human action needed.
ADVISORY	advisory	Human FYI, "you should know".
CASCADE	cascade	Top-down mandatory push (parent → child).
ESCALATION	escalation	Bottom-up authority request (child → parent).
HANDOFF	handoff	Ownership transfer.
REQUEST_SUPPORT	request-support	Peer asks peer for help (patrols, aircraft, expertise).
COORDINATION	coordination	Joint action, shared ownership.
OBSERVER_ADD	observer-add	Loop someone in without giving them action authority.
OBSERVER_PROMOTE	observer-promote	Observer requests actor status.
SIT_REP	situation-report	Downstream → upstream status update.

IF-6.6 Interaction matrix

Which flows are allowed for each relationship class. Exported as `FLOW_MATRIX` mapping `relationship class → Set<flow type>`.

Relationship (from A to B)	Allowed flows
self	situation-report (log-only)
parent (A parent of B)	notification, advisory, cascade, handoff, request-support, coordination, observer-add
child (A child of B)	notification, advisory, escalation, handoff, request-support, coordination, observer-add, situation-report
sibling	notification, advisory, handoff, request-support, coordination, observer-add
same-branch (non-sibling)	notification, advisory, request-support, coordination, observer-add (no handoff without escalation)

Relationship (from A to B)	Allowed flows
cross-branch	notification , advisory , handoff , request-support , coordination , observer-add (no cascade)
unknown	none

Enforcement.

```
canInitiate(fromRoleId, toRoleId, flowType) -> {allowed: boolean, reason?: string}
```

The gate applies three layers.

1. Admin bypass. `admin` -kind roles are allowed everything.
2. Operator wide push. `operator` -kind roles can push any flow to any receiver or operator (they own site-scoped escalation).
3. Receiver-to-receiver check against `FLOW_MATRIX`.

Rejections return a machine-readable `reason` string so callers can log or surface why.

IF-6.7 Path taxonomy for events

The set of roles impacted by a given event at time `t` is the union of six sources.

1. **Direct scope.** Roles whose `siteScope` (from `SITES[siteId].receivers`, IF-6.8) includes the event's `siteId`. Auto-notified on every lifecycle transition.
2. **Cross-site cascade.** When `_spawnLinkedSiteEvent` creates a shadow event at another site, roles scoped to that site are auto-notified. If the primary event is `hostile`, cross-scope advisory upgrades apply within the same top-level agency branch.
3. **Threat-type routing.** `(siteType × platform × classification)` matrix (in `routing.js`, planned) auto-adds observer or actor roles. Example: `substation × quadcopter × hostile` adds Energinet DSO as actor and Beredskab as observer.
4. **Explicit chain.** Any role added via `handoff`, `cascade`, or `observer-add` interaction on this event.
5. **Escalation ancestors.** When a role escalates to its parent, parent auto-added as observer.
6. **Coordination peers.** `coordination(withRole)` adds that peer as actor with shared ownership.

Query helper.

```
impactedRoles(event, siteReceiversLookup) -> Array<{role_id, mode, addedBy, reason, tier?, cond
```

Phase 0 populates from direct scope only when `siteReceiversLookup` is provided. Phase 1+ extends with the other five sources.

IF-6.8 Site-scoping schema (planned)

`SITES[siteId].receivers` is the pluggable block that declares which government profiles are in-scope for a site. Not wired yet. Target shape:

```
SITES[siteId].receivers = [
  { id: 'rcv-pol-kbh',          mode: 'actor',    role: 'primary-response',  tier: 1 },
  { id: 'rcv-brs-hedehusene',   mode: 'actor',    role: 'emergency-response', tier: 2 },
  { id: 'rcv-forsvar-fly-skrydstrup', mode: 'actor', role: 'air-response',      tier: 3 },
  { id: 'rcv-pol-hq',           mode: 'observer' },
  { id: 'rcv-pol-special-pet',   mode: 'observer' },
  { id: 'rcv-region-hst',        mode: 'observer', condition: 'casualty-scenario' },
]
```

Field	Type	Notes
<code>id</code>	string	Role id from IF-6.2.
<code>mode</code>	enum	<code>actor</code> (has CTAs) or <code>observer</code> (in-the-loop only).
<code>role</code>	string	Response role name for playbook selection (<code>primary-response</code> , <code>air-response</code> , <code>regulator</code> , <code>municipal-crisis</code> , <code>observer</code> , ...).
<code>tier</code>	number	Escalation tier bucket (1-4).
<code>condition</code>	string	Optional rule id from <code>routing.js</code> . Auto-adds this observer only when the condition matches (e.g. <code>casualty-scenario</code> , <code>cbrn-threat</code> , <code>mass-casualty</code> , <code>nuclear-threat</code> , <code>sustained-incident</code>).

IF-6.9 Actor versus observer mode

Two participation modes per event per role, tracked on `event.participants` .

```
event.participants: Map<role_id, {
  mode: 'actor' | 'observer',
  addedAt: ISO 8601,
  addedBy: role_id,
  addReason: string,
  promotedFromObserver: boolean,
  ackedAt: ISO 8601 | null,
  ackedBy: role_id | null,
}>
```

Actor mode: CTAs render, actions are recorded to the escalation log, this role appears in the "assigned responders" list.

Observer mode: notifications delivered, CTAs hidden, "OBSERVER" chip on the case card.

The `observer-promote` flow requests actor status. Any current actor can approve.

`event.interactions` records every flow between roles for compliance auditing.

```
event.interactions: Array<{
  id: string,
  timestamp: ISO 8601,
  flow: FLOW_TYPES value,
  from_role_id: string,
  to_role_id: string,
  payload: object,
  ackStatus: 'pending' | 'acked' | 'rejected',
  ackedAt: ISO 8601 | null,
  ackedBy: role_id | null,
}>
```

IF-6.10 API surface

Full public API from `roles.js`.

Function	Returns	Purpose
<code>getRole(id)</code>	RoleDefinition or null	Registry lookup.
<code>allRoles()</code>	RoleDefinition[]	Full registry, all kinds.
<code>receiverRoles()</code>	RoleDefinition[]	Receivers only.
<code>operatorRoles()</code>	RoleDefinition[]	Operators only.
<code>getRoleChildren(roleId)</code>	RoleDefinition[]	Direct leaf children of a parent role.
<code>getRoleDestinationIdsRolledUp(roleId)</code>	string[]	All destination ids under this role, recursively.
<code>agencyBranchOf(roleId)</code>	AGENCY_BRANCHES value or null	Top-branch resolution.
<code>relationshipBetween(aId, bId)</code>	relationship class string	Pair classification.
<code>canInitiate(fromId, toId, flowType)</code>	<code>{allowed: bool, reason?: string}</code>	Permission gate.
<code>impactedRoles(event, siteReceiversLookup?)</code>	Array of <code>{role_id, mode, addedBy, reason, ...}</code>	Direct-scope receivers for an event. Extended in Phase 1+.
<code>getActiveRole()</code> / <code>setActiveRole(id)</code> / <code>onRoleChange(fn)</code>	active role state	UI role switcher plumbing.

Test hook: all helpers are exposed on `window.__isrRoles` for browser console debugging.

IF-6.11 Onboarding checklist for a new receiver

Adding a new role is a data change. Zero code changes required for the runtime path.

1. Add row to `RECEIVERS` in `roles.js` with `id`, `kind: 'receiver'`, `type` (leaf or parent), `org`, `label`, `initials`, `scope`, `parentId` (or `childrenIds`), `description`, and empty `destinationIds`.
2. Add site references in `SITES[siteId].receivers` blocks for every site this role covers.
3. If auto-escalation is required, add matching entries in `destinations.js` and mark the site's `autoEscalateTiers`.
4. If the role has a distinct response playbook (per `siteType × platform × classification`), add entries in `runbooks.js`.
5. Add UI copy for inbox card, toast text, escalation log labels.
6. Smoke test: fire a scenario at a covered site, confirm this role appears in `impactedRoles(event)`, receives the notification, and (for actors) has the correct CTAs.

The runtime path (marker engine, event lifecycle, coverage helpers, sensor POV, Cesium rendering) is untouched by any of the above.

IF-6.12 Migration path

The receiver contract is rolling out in phases so each ship is small and verifiable. Every phase is additive.

Phase	Contents	Status
Phase 0	Full registry (242 leaves) + <code>agencyBranchOf</code> + <code>relationshipBetween</code> + <code>canInitiate</code> + <code>impactedRoles</code> . Verifiable via <code>window.__isrRoles</code> .	<code>[live]</code> (commits <code>cd24117</code> + <code>410a68a</code> + <code>1b9b909</code>)
Phase 1	<code>event.participants</code> + <code>event.interactions</code> + <code>event.routingHistory</code> fields. <code>SITES[siteId].receivers</code> schema wired into <code>impactedRoles</code> . <code>pushNotification</code> wrapper alongside existing <code>toast()</code> calls. Cross-site shadow-spawn advisory (P0 audit item) fires automatically. Verifiable via <code>window.__isrPhase1</code> .	<code>[live]</code> (commits <code>b78fcfc</code> + <code>35449a8</code>)
Phase 2	<code>availableCTAsForReceiver(role_id, event, ctx)</code> factory replaces the universal CTA list. Role-branch scoped action palette (Politi → patrol + cordon + AKS, Forsvaret → fighter + army, BRS → standby + deploy, Trafik → NOTAM, Kommune → crisis staff + shelter, Region → ambulance + triage, HJV → reinforce). Observer chip in receiver report header when mode=observer. Full-viewport observer picker overlay with client-side substring search over all 242 profiles. Router stubs for all new actions log to <code>event.interactions</code> even where the real dispatch backend lands in Phase 3. Verifiable via <code>window.__isrPhase2</code> .	<code>[live]</code> (commits <code>d76b38e</code> + <code>2f24e8e</code>)
Phase 3	Remaining audit patches on the new contract. Cross-site backlink chip in receiver cards. "Primary still active at X" surface on linked-event closure. Substation-specific playbook + Energinet DSO CTA in post-incident handoff. Energinet night-mode rendering. siteld validation against SITES keys. Real dispatch pipeline for Phase 2 CTAs (patrol dispatch, NOTAM push, ambulance service).	<code>[planned]</code>
Phase 4	Full flow palette in UI (<code>request-support</code> button between peer receivers, <code>coordination</code> workflow, <code>escalation</code> upward).	<code>[planned]</code>

Every phase can ship without the next. Phase 0 is complete and verifiable today. Phase 1 begins after the current marker regression pass lands green.

IF-7. Response and dispatch

Status: pending. Internal coordination surface for the responding authority. Will specify the asset catalog, playbook and runbook schemas, the dispatch object, and the interceptor state machine.

IF-8. Recording and evidence export `[partial]`

Two subsections written so far: the trajectory-scoping contract for the debrief, replay, and evidence-export surfaces (IF-8.1). The full recording sample schema, shadow-fallback fetch contract, and evidence bundle format are still pending.

IF-8.1 Trajectory scoping per viewer role `[live]`

Different roles see different portions of a threat trajectory when opening the debrief, replay, or downloading JSON / CSV evidence. Scope is derived from the viewer's role kind + owned-site scope.

Rules.

Viewer role kind	What they see
Admin	Full trajectory across every site. All segments solid. No scope banner.
Receiver (state agency: Politi, PET, FE, Forsvaret, BRS, Region, kommune, HJV, ministry, agency)	Same as admin — full access for now. Reserved for tightening in a later phase if a customer restricts an agency's cross-site visibility.
Operator (owns 1 site the threat crossed)	Trajectory samples inside owned site's sensor coverage only, rendered solid. All other segments hidden. Scope banner: "Trajectory scoped to your site perimeter. State agencies see the full path."
Operator (owns 2+ sites the threat crossed)	Confirmed segments per owned site + INFERRED (dotted) bridges connecting consecutive confirmed segments. Segments outside all owned sites hidden. Scope banner: "Segments between your sites are inferred (dotted) — your sensors did not observe them directly."
Operator (owns 0 sites the threat crossed)	Empty. Toast + banner: "Event outside your site scope." Replay aborts before rendering. Export writes zero rows and warns.

Entry point.

```
scopedTrajectoryFor(event, roleId, samples) -> {
  segments: Array<{ positions: Sample[], visibility: 'confirmed' | 'inferred', droneId: string
  fullyVisible: boolean,
  hiddenSegmentCount: number,
  scopeNote: string | null,
  ownedSiteIds: string[] | null,
  _passthrough: boolean
}
```

`_passthrough: true` signals full-access — legacy per-drone + 800 m gap-split renderer runs unchanged. `_passthrough: false` returns pre-scoped segments ready for direct render.

Site containment.

```
_findSiteContainingPoint(lat, lon) -> siteId | null
```

Walks `SITES` online sensor coverage circles (per IF-2.4) and returns the first site whose union of circles contains the point. Null if outside all site coverage.

Confirmed vs inferred segments.

- `confirmed` — every sample is inside an owned-site sensor cov. Rendered as a solid polyline with confidence-band colour.
- `inferred` — synthetic 2-point segment connecting the last confirmed sample of one run to the first confirmed sample of the next. Rendered as dashed line, dim alpha, thinner width. Only emitted when the operator owns MULTIPLE sites (single-site operators see no inferred bridges).

Where the contract applies.

Surface	Function	Contract behaviour
Debrief map trajectory	<code>_debriefRenderTrajectory(samples, event)</code>	Operator sees scoped segments; admin/receiver sees legacy per-drone + gap-split render.
Debrief narrative panel	<code>_debriefBuildNarrativePanel(event, ...)</code>	Prepends a warn-toned "Scoped view" banner with <code>scopeNote</code> and <code>hiddenSegmentCount</code> when scope is restricted.

Surface	Function	Contract behaviour
Replay overlay	<code>startReplay(eventId)</code>	<code>droneEntries</code> filtered to confirmed samples. Multi-site operators additionally get inferred-bridge polylines. Empty-scope aborts replay.
JSON evidence export	<code>window.__isr_downloadRecording(eventId)</code>	Wraps raw recording through <code>_scopeRecordingForActiveRole</code> . Downloaded JSON carries <code>_scopeMeta</code> field describing the scope applied.
CSV evidence export	<code>window.__isr_downloadRecordingCSV(eventId)</code>	Same scoping. Aborts + toasts on empty-scope condition to avoid downloading a blank CSV that reads as broken data.

Scope metadata on exports.

Every scoped export carries `_scopeMeta` in the JSON envelope:

```
{
  scopedForRole: string,
  fullyVisible: false,
  hiddenSegmentCount: number,
  ownedSiteIds: string[],
  scopeNote: string,
  originalSampleCount: number,
  scopedSampleCount: number
}
```

Downstream analyst tooling (Excel, Python, R) can detect scope via this field. Full-access exports omit `_scopeMeta` entirely.

Safety defaults.

- Missing or unresolvable `roleId` → treated as full-access. Prevents accidental data suppression on caller error.
- Missing `event` argument to `_debriefRenderTrajectory` → falls through to legacy path, unscoped. Existing call sites that don't pass event stay backward-compatible.
- Unknown role kind → full-access with a `console.warn`. Fail-open on scope errors.

What's not yet wired.

- PDF evidence report — same scoping contract will apply when it lands.
- Receiver-side inbox pre-listing (an operator viewing the receiver dashboard shouldn't see cross-scope events in the listing, only in the workspace view). Deferred to a follow-up commit.
- Sub-site scoping (an operator with multiple sensors within one site limiting to specific sensor coverage) — not required for v1.

IF-9. Agentic (Mistral) interfaces `[partial]`

IF-9.1 Design principle and where the layer sits

The pipeline is deterministic where a response decision depends on it and generative where an analyst adds narrative nuance. Correlation scoring, recommendation, escalation routing, and dispatch are deterministic. The model-backed layer produces natural-language narrative and reasoning only. It never sits on the dispatch or escalation critical path.

If the model endpoint is unreachable, slow, or unconfigured, the caller keeps its already-rendered deterministic text. The model narrative replaces that text only when it arrives. A partner integrating here must treat every model output as best-effort enrichment, never as a gate.

Agent B consumes the same source-agnostic detection stream as everything else (IF-1). Whether a detection came from the mock source or a real field node is invisible here, so the narrative reads the same in a demo and in production.

Two agents are model-relevant. Agent A is deterministic today and target-generative. Agent B is generative today via Mistral.

Agent	Role	Model status
Agent A — Site Context	Per-site knowledge base. Frame of reference for every narrative.	<code>[live]</code> deterministic (hand-curated baseline). <code>[planned]</code> Mistral-generated per site.
Agent B — Correlation and Narrative	Reads a detection against Agent A context and produces the analyst briefing.	<code>[live]</code> Mistral streaming, with deterministic mock fallback.

IF-9.2 Agent A — Site Context `[live: deterministic]`

Agent A maintains a per-site context object, cached and loaded offline. It is the frame of reference Agent B narrates against. Source is `site_context.js`.

Entry point. `contextForSite(siteId) -> SiteContext` .

SiteContext shape.

Field	Type	Notes
<code>site_id</code>	string	Site slug.
<code>name</code>	string	Display name.
<code>site_type</code>	string	For example <code>commercial_airport</code> , <code>hv_switching_station</code> .
<code>schema_version</code>	string	Currently <code>1.2</code> .
<code>generated_at</code>	ISO 8601	
<code>generated_by</code>	string	Currently <code>Agent A stub</code> . Target: Mistral model version.
<code>airport_reference_point</code>	<code>{lat, lon}</code>	Optional. Airports only.
<code>critical_areas</code>	object[]	<code>{id, name, center:{lat,lon}, criticality, verified, source, reason, status?}</code> .
<code>high_value_assets</code>	object[]	<code>{id, name, location:{lat,lon}, asset_type, criticality, reason}</code> .
<code>response_asset_positions</code>	object[]	<code>{id, name, location, asset_type, criticality, reason}</code> .
<code>aircraft_of_interest</code>	object[]	<code>{id, callsign, operator, aircraft, registration, stand, location, status, eta_departure, route, manifest_note}</code> .
<code>sensitive_setups</code>	object[]	<code>{id, name, note}</code> .
<code>normal_patterns</code>	object	<code>{operating_hours, approach_corridors[], maintenance_windows, civilian_drone_activity, routine_helicopter_activity}</code> .

Field	Type	Notes
<code>correlator_hints</code>	object	<code>{dwell_alarm_zones:[{name, center, radius_m, threshold_sec}], unusual_pattern_flags:[string]}</code> .

Helpers for Agent B.

Helper	Signature	Returns
<code>nearestCriticalArea</code>	<code>(siteId, lat, lon, maxKm)</code>	Closest critical area within <code>maxKm</code> , or null.
<code>dwellZonesAtPoint</code>	<code>(siteId, lat, lon)</code>	The dwell alarm zones a point falls within.

Current state. The context is a hand-curated baseline for `cph`, the five Energinet substations, and `esbjerg`, with stubs for the remaining sites. `generated_by` is `Agent A stub`. The target is a model pass that generates and refreshes this per site on any site-definition change. Until then, a partner treats SiteContext as static reference data loaded at boot.

IF-9.3 Agent B — Correlation and Narrative [live]

Agent B reads a detection against Agent A context and produces a two-part analyst output: a briefing body and a one-sentence recommendation. Two entry points, both streaming, both in `mistral.js`.

Entry point	When	Inputs consumed
<code>streamCaseFileNarrative(event, site, callbacks)</code>	Live event, case-file view	Event identity, <code>event.subject</code> (DetectionSubject digest), site name, threat and classification buckets, count of correlated events.
<code>streamDebriefNarrative(event, samples, analysis, callbacks)</code>	Closed event, post-event debrief	DetectionSubject digest at close, trajectory dwell profile, closest-asset approach, and the class-change log.

Output shape (delivered through `onDone`):

```
{
  body: string,           // briefing, under 500 chars, max two paragraphs
  recommendation: string, // one sentence, action verb, under 200 chars
  model_version: string   // the model id that produced it
}
```

The DetectionSubject the narrative is grounded in is specified in IF-4. Agent B is told to trust the labeled subject fields as fused NN output and not confabulate around missing ones.

IF-9.4 Mistral client interface [live]

The client is a single streaming call over server-sent events. Source is `mistral.js`.

Property	Value
Endpoint	<code>https://api.mistral.ai/v1/chat/completions</code>
Model	<code>mistral-large-latest</code>
Transport	HTTPS POST, <code>Accept: text/event-stream</code> , bearer token
Streaming	SSE. Each <code>data:</code> line is a JSON chunk. Content read from <code>choices[0].delta.content</code> . Terminator line is <code>data: [DONE]</code> .

Property	Value
Request body	<code>{model, messages, stream: true, temperature, max_tokens}</code>
<code>temperature</code>	default 0.2
<code>max_tokens</code>	default 800
Timeout	30000 ms, enforced by an abort controller
Auth token	<code>VITE_MISTRAL_API_TOKEN</code>
Configured check	<code>isMistralConfigured()</code> returns true when a token is present

Callback contract. The public entry points wrap the raw stream in a delimited handler:

Callback	Fires
<code>onBodyDelta(text, replace)</code>	Body text arrives. <code>replace</code> true means the handler re-emits the full running body so far.
<code>onRecoDelta(text, replace)</code>	Recommendation text arrives, after the delimiter is seen.
<code>onDone({body, recommendation, model_version})</code>	Stream complete.
<code>onError(err)</code>	Any failure. Caller keeps its mock text in place.

Delimiter protocol. The model is instructed to emit the briefing body, then a line `===RECO===`, then the one-sentence recommendation. The stream handler switches from body phase to recommendation phase when it sees the delimiter, so the two fields render into separate UI targets during streaming.

IF-9.5 Prompt contract `[live]`

Both entry points build a system prompt plus a user prompt. The house style is enforced in the prompt, not only in post-processing.

Writing rules injected into every system prompt:

- Write in English. Use Danish characters æ, ø, å only for proper nouns such as site or agency names, never for body text.
- Two short paragraphs maximum for the body.
- Declarative voice. No hedging such as "may" or "could indicate".
- No em-dashes. No semicolons. No filler such as "furthermore" or "moreover".
- No markdown. No bold, italic, headers, or bullets.
- Body under 500 characters.
- Recommendation is one sentence, starts with an action verb, under 200 characters.

Strict output format:

```
<briefing body here>
===RECO===
<one-sentence recommendation here>
```

The user prompt carries the DetectionSubject digest (IF-4) plus, for debriefs, a trajectory-analysis block (dwell profile, closest asset approach) and a classification-log block (class transitions during the event).

IF-9.6 Failure modes and fallback [Live]

Failure	Behavior
Token not configured	<code>isMistralConfigured()</code> is false. Entry points call <code>onError</code> immediately. Caller keeps deterministic mock text.
Endpoint unreachable or non-200	<code>onError</code> with the HTTP status. Mock text stays.
Timeout (30 s)	Abort controller fires, <code>onError</code> . Mock text stays.
Malformed SSE chunk	That chunk is ignored, stream continues.

The deterministic path is never blocked by this layer. This is the single most important property for a partner to preserve when they re-home the endpoint.

IF-9.7 Target cloud architecture (Azure) [planned]

The target moves model inference off the browser and onto Azure, behind a sovereign proxy that removes the browser-side token. The client contract in IF-9.4 does not change. Only the endpoint and the auth boundary move. This subsection supersedes earlier references to Scaleway or OVH hosting. Azure is the intended host.

Facts below were verified against Microsoft documentation on 2026-08-23. This is a fast-moving catalog on a 90-day update cycle. Confirm live in the Foundry portal at deploy time. Sources are listed at the end of this subsection.

Product naming. The platform was Azure AI Studio, renamed Azure AI Foundry (November 2024), then Microsoft Foundry (November 2025). The model catalog is Microsoft Foundry Models, split into "sold by Azure" and "partners and community". Mistral sits in the partners-and-community category and is billed through Azure Marketplace. Two deployment mechanisms exist: serverless deployment (pay-per-token API, formerly Models-as-a-Service, no GPU quota) and managed compute (dedicated virtual machines on Azure Machine Learning).

Model version. The live client sends `mistral-large-latest`, which is a Mistral-API alias and does not exist as an Azure SKU. On Azure you pin a specific catalog SKU. Note `mistral-large-2411`, referenced in the older architecture note, was retired on Azure on 2026-01-30. Current serverless Mistral SKUs include `Mistral-large (v1)`, `mistral-medium-2505` (Mistral Medium 3.5), `mistral-small-2503`, `Codestral-2501`, `Ministral-3B`, and `Mistral-Large-3` in preview. Pin one of these at deploy and stamp it as `model_version` on every generated artifact.

Component mapping.

Platform need	Azure service	Note
Model inference	Microsoft Foundry Models, Mistral serverless deployment (or managed compute)	EU serverless regions include France Central and Sweden Central.

Platform need	Azure service	Note
Sovereign proxy (removes browser token)	Azure Container Apps or Functions behind Azure API Management	Holds the key server-side. Browser calls the proxy, not <code>api.mistral.ai</code> . Streams SSE straight through.
Event and evidence persistence	Azure Blob Storage with immutability (WORM) policy, optional customer-managed key	Signed evidence hashes and chain of custody.
Correlation index	Azure AI Search	Cross-site correlation index.
Identity and tenant isolation	Microsoft Entra ID, with subscription and resource-group boundaries per tenant	Enforces the three-tenant isolation from Section 1.
Network isolation	Azure Private Link and Private Endpoints, VNet integration	Private inference endpoint, no public exposure.
Deployment-SKU governance	Azure Policy	Restrict deployments to approved EU regions and SKUs.

Data residency posture. Three separate axes, do not conflate them.

- Data at rest stays in the designated Azure geography. Choosing EU regions keeps at-rest data in Europe.
- Inference processing residency is the caveat. Serverless Mistral is offered as Global Standard only today, and Global Standard may process prompts and responses in any region where the model is deployed. Data Zone Standard, which would pin processing to the EU Data Boundary, is currently listed as "Not available" for Mistral models. Strict EU-only processing residency therefore requires one of: managed compute (dedicated in-region GPU), a later addition of Data Zone support for Mistral, or the on-premises path below. This must be verified in-portal before any residency claim is made to a customer.
- On-premises and disconnected path. Foundry Local on Azure Local runs models locally under Microsoft Sovereign Private Cloud, which Microsoft markets specifically for critical infrastructure and national security. This is the fallback when public-cloud processing residency is insufficient. It removes the processing-residency caveat by keeping inference in-country or fully disconnected.
- Training and sharing. Microsoft acts as data processor. Per Microsoft documentation, prompts and outputs are not used to train Microsoft, model-provider, or third-party models, models are stateless and do not store prompts or outputs, and prompts and outputs are not shared with the model provider. Commercial and usage metadata may be shared with the provider for billing and contact.

Broader sovereignty envelope. The Microsoft EU Data Boundary was completed on 2025-02-26 and covers Azure, keeping customer data and pseudonymized personal data stored and processed in the EU and EFTA (EU 27 plus Iceland, Liechtenstein, Norway, Switzerland). Denmark is a named boundary country with a Danish datacenter, so a standard EU-region deployment already places customer data and pseudonymized logs in the EU. On top of that, Microsoft Sovereign Cloud (announced 2025) offers two tiers relevant here. Sovereign Public Cloud adds customer-managed keys, External Key Management, and Data Guardian (access controlled by Europe-based personnel). Sovereign Private Cloud, delivered through Azure Local and Foundry Local, is the customer- or partner-operated tier Microsoft positions for defence and critical infrastructure. One honest caveat: the strongest of these controls (Data

Guardian, the full EU AI-processing commitment, disconnected operation) are still rolling out across late 2025 into 2026 and are not all GA. Verify the service matrix at contract time rather than assuming it is live.

Migration from the live state. Today the browser calls `api.mistral.ai` directly with `VITE_MISTRAL_API_TOKEN` baked into the build, which exposes the key and is dev and demo only. The target has the browser call the ISR sovereign proxy on Azure. The proxy holds the key, calls the Foundry endpoint, and streams SSE back unchanged. The IF-9.4 client contract stays identical from the browser's point of view. Only `ENDPOINT` and the auth header move server-side.

Sources.

- Microsoft Foundry deployment types and data processing: <https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/deployment-types>
- Foundry Models from partners and community (Mistral regions and residency): <https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-from-partners>
- Model catalog data privacy (no training, processor role): <https://learn.microsoft.com/en-us/azure/ai-foundry/how-to/concept-data-privacy>
- EU Data Boundary: <https://learn.microsoft.com/en-us/privacy/eudb/eu-data-boundary-learn>
- Microsoft Sovereign Cloud: <https://learn.microsoft.com/en-us/azure/azure-sovereign-clouds/microsoft-sovereign-cloud>
- Mistral Large 2411 retirement on Azure (2026-01-30): <https://docs.mistral.ai/models/mistral-large-2-1-24-11>

IF-10. Runtime API, DOM mount, external services

Status: pending. Will specify the `window.__isr_*` surface, the DOM mount contract, external service endpoints, and the environment surface.

Appendix

Status: pending. Data dictionary, full enums, ID formats, coordinate conventions, and the open-gaps register.